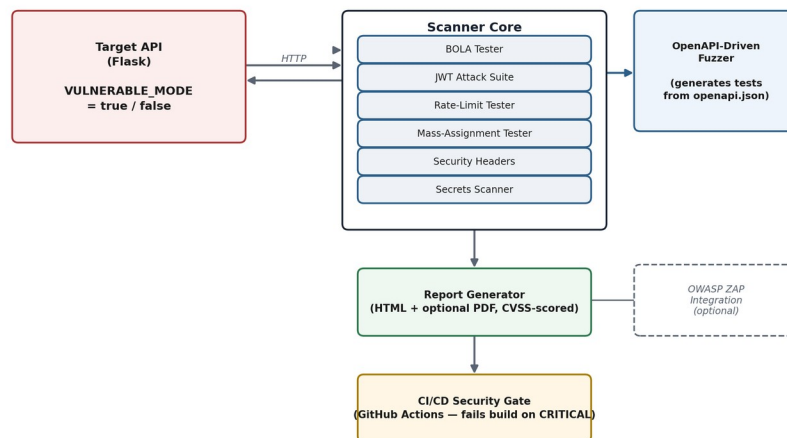


A PROJECT REPORT ON

API Security Testing Framework

Automated Vulnerability Detection and Remediation for the OWASP API Security Top 10 (2023)

API Security Testing Framework — System Architecture



Submitted by

Bharath | [Roll / Register Number]

[Course / Programme Name]

[Department Name]

[Institution Name]

August 2026

Abstract

This project presents the design and implementation of an automated security testing framework for REST APIs, built around the OWASP API Security Top 10 (2023) standard. The framework consists of two cooperating components: a deliberately vulnerable Flask target API whose behaviour is controlled by a single `VULNERABLE_MODE` flag, and a modular Python scanner that probes the target for Broken Object Level Authorization (API1), Broken Authentication (API2), Broken Object Property Level Authorization (API3), Unrestricted Resource Consumption (API4), Broken Function Level Authorization (API5), and Security Misconfiguration (API8), in addition to a cross-cutting secrets-exposure check.

Rather than hardcoding test cases per endpoint, the scanner includes an OpenAPI-driven fuzzing engine that reads the target's `openapi.json` specification and automatically generates authorization and mass-assignment test cases from it. Findings are aggregated into a severity-scored HTML report with remediation guidance, and the same scan is wired into a GitHub Actions workflow that fails a pull request build whenever a CRITICAL finding is present, turning the scanner from a manual script into a continuous security gate.

Every vulnerability seeded in the target was independently exploited by hand (via curl and short Python snippets) before being confirmed by the automated scanner, and the same scan was re-run against the hardened (`VULNERABLE_MODE=false`) configuration to confirm remediation. The final validated scan reports 14 findings (10 CRITICAL) against the vulnerable configuration and 0 findings against the hardened configuration, across every module. During development, a gap was found where one scanner check (missing-signature-verification) had no corresponding seeded bug; the target was corrected and the fix was re-validated, illustrating an iterative detect → root-cause → remediate → confirm workflow that mirrors real application security practice.

Table of Contents

Abstract	2
1. Introduction	3
2. Related Work and Existing Tools	4
3. System Architecture	4
4. Technology Stack	6
5. Implementation	6
6. Results and Demonstration	9
7. Testing and Validation	18
8. Results Summary	19

9. Conclusion and Future Scope 20

10. References 21

1. Introduction

1.1 Motivation

Application Programming Interfaces (APIs) are now the primary attack surface for modern web and mobile applications: nearly every client-side feature is backed by an API call, and unlike a browser-rendered page, an API endpoint will typically do exactly what it is asked, with no UI to hide a missing permission check. The Open Web Application Security Project (OWASP) formalised this risk landscape in its API Security Top 10, most recently updated in 2023, which catalogues the ten most critical and most commonly exploited classes of API vulnerability observed across real-world engagements.

Most student and portfolio security projects stop at demonstrating a single vulnerability by hand. This project instead asks: what would it take to build a small, honest version of the kind of automated scanner a security engineering team would actually use — one that detects a recognised set of vulnerability classes, explains why each finding matters, and can be dropped into a CI/CD pipeline as a quality gate?

1.2 Problem Statement

Given a REST API, manually verifying that every endpoint correctly enforces authentication, authorization at the object level, authorization at the property level, function-level access control, rate limiting, and safe error handling is slow and error-prone, and the check is easy to skip once a deadline is close. The problem this project addresses is: design a target-agnostic scanner that can automatically detect representative instances of six OWASP API Security Top 10 (2023) categories against a running API, generate an actionable report, and gate a CI pipeline on the result — and prove that it works using a target whose vulnerabilities are known in advance.

1.3 Objectives

- Implement a realistic target API with an explicit, single-flag toggle between an insecure and a hardened configuration on the same routes.
- Implement independent, pluggable scanner modules for BOLA, broken authentication (JWT attacks), unrestricted resource consumption, mass assignment, broken function-level authorization, security misconfiguration, and secrets exposure.
- Implement an OpenAPI-driven fuzzing engine that generates test cases from a specification file rather than a hardcoded endpoint list.
- Generate a severity-scored, remediation-annotated HTML report from scan results.
- Integrate the scanner into a CI/CD pipeline (GitHub Actions) as a pass/fail security gate.
- Validate every seeded vulnerability by manual exploitation, confirm the automated scanner detects the same issue, and confirm the hardened configuration reports zero findings.

1.4 Scope

The framework is designed to be run against a target the operator owns or is explicitly authorized to test — in this project, exclusively the bundled local target API. OWASP ZAP integration, a scan-history dashboard, PostgreSQL persistence, and Slack/email alerting were designed for and are documented as extension points (Section 9.2) but were intentionally left out of the core deliverable to keep the primary scanner focused and fully validated within the available time.

2. Related Work and Existing Tools

The framework draws on ideas from several established categories of security tooling rather than inventing a new approach from scratch:

- OWASP ZAP (Zed Attack Proxy) — a general-purpose, widely used web/API active scanner. This project integrates with ZAP's API (scanner/zap_integration.py) as an optional supplementary engine rather than reimplementing active scanning.
- TruffleHog / GitLeaks — secret-scanning tools that combine regular-expression pattern matching with Shannon-entropy analysis to flag likely credentials. The same technique is applied here to live HTTP responses and headers (scanner/modules/secrets_scanner.py) rather than git history.
- Postman/Newman and schema-driven fuzzers — tools that can replay a collection or a schema against an API. This project's OpenAPI fuzzer (scanner/fuzzer/openapi_fuzzer.py) applies the same schema-driven idea specifically to authorization testing, which general-purpose fuzzers do not typically target out of the box.
- OWASP's own deliberately vulnerable applications (e.g. WebGoat, crAPI, DVWA) — the precedent for pairing a scanner with a known-vulnerable target so that detection can be objectively validated, which is the same strategy this project uses at a smaller scale.

The contribution of this project is not a novel detection technique, but the combination of a purpose-built, dual-mode target with a modular scanner and an auto-generating fuzzer in one small, fully working, and fully tested codebase — something that is straightforward to read end to end, unlike the much larger codebases of the production tools above.

3. System Architecture

The system is deliberately split into two codebases that only ever communicate over HTTP, mirroring how a scanner and a real target relate in practice: the scanner never imports target code, and the target has no knowledge of the scanner. Figure 1 shows the overall architecture.

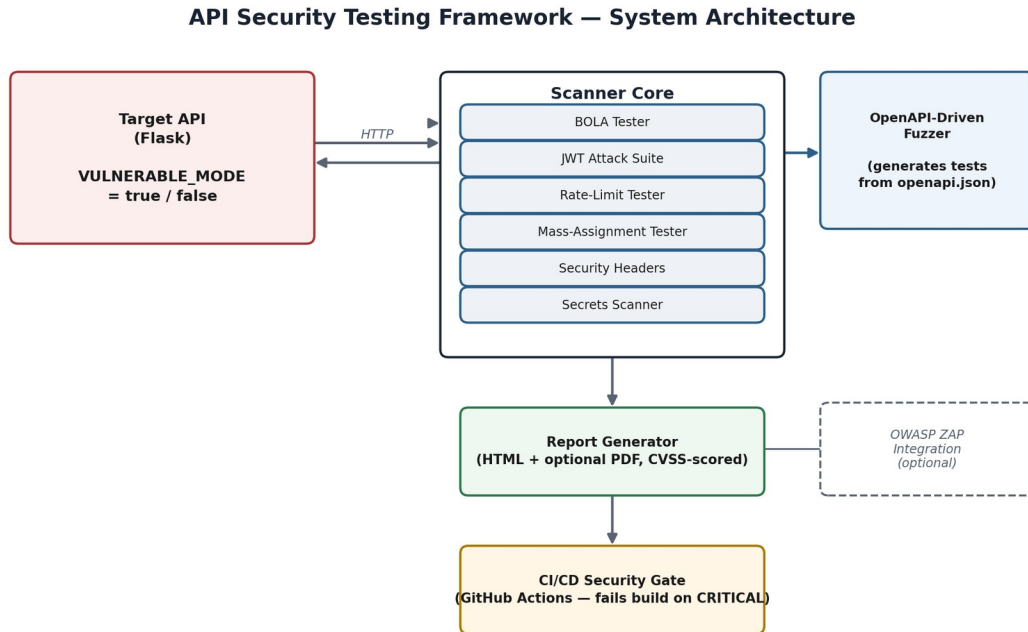


Figure 1. High-level architecture: the target API exposes an OpenAPI spec and REST endpoints; the scanner core runs six independent modules plus an OpenAPI-driven fuzzer against it, aggregates results into a report, and that report's pass/fail outcome gates CI.

3.1 Design Principle: the `VULNERABLE_MODE` Toggle

Every seeded vulnerability in the target API is guarded by a single environment variable, `VULNERABLE_MODE`. Each affected route contains both the insecure code path (used when the flag is true) and the correct fix (used when it is false), on the same function, so the two configurations can never drift apart into two different codebases. This is what allows the same scanner run to be repeated before and after remediation as direct, comparable evidence — Section 6 and Section 8 rely on exactly this comparison.

3.2 Component Overview

Table 1. Component responsibilities within the framework.

Component	Responsibility
target_api/	Deliberately vulnerable Flask API (accounts, orders, admin, login) with the <code>VULNERABLE_MODE</code> toggle
scanner/modules/	Six independent, pluggable vulnerability-detection modules
scanner/fuzzer/	OpenAPI-driven engine that auto-generates test cases from <code>openapi.json</code>
scanner/report_generator.py	Aggregates findings, computes severity summary, renders HTML/PDF
scanner/cli.py	Entry point; wires modules + fuzzer + report together; sets the CI exit code

Component	Responsibility
scanner/zap_integration.py	Optional OWASP ZAP active-scan client (Section 9.2)
.github/workflows/	GitHub Actions workflow that runs the scanner on every pull request

4. Technology Stack

Table 2. Technology stack by layer.

Layer	Technology
Target API	Flask 3.0, Python 3.11+
Authentication	PyJWT 2.9 (HMAC-SHA256)
Password hashing	Werkzeug security helpers
Data layer	In-memory store (see Section 9.2 for a PostgreSQL extension path)
Scanner HTTP clients	requests (synchronous), httpx (async, for concurrent flood testing)
Concurrency	Python asyncio, for the rate-limit flood test
Reporting	Jinja2 (HTML templating); WeasyPrint (optional PDF export)
Testing	pytest, integration-style against a live in-process target
Containerization	Docker, docker-compose
CI/CD	GitHub Actions

5. Implementation

5.1 Vulnerable Target API

The target exposes a small, realistic domain — user accounts with balances and orders, rather than generic to-do items — so that a finding such as “user A can read user B’s balance” demonstrates real impact. Six routes are affected by `VULNERABLE_MODE`, one per OWASP category covered.

5.1.1 API1:2023 — Broken Object Level Authorization

The account and order endpoints skip the ownership check entirely when `VULNERABLE_MODE` is true:

```
@app.route("/api/orders/<int:order_id>")
@auth_required()
def get_order(order_id):
    order = db.get_order(order_id)
    if order is None:
        return jsonify({"error": "not found"}), 404
    if not VULNERABLE_MODE and order.user_id != g.user_id:
        return jsonify({"error": "forbidden"}), 403
    return jsonify({"id": order.id, "user_id": order.user_id,
```

```
"item": order.item, "amount": order.amount})
```

The route only ever checks that the caller holds a valid token — not that the token's owner is the resource's owner. The ownership check exists but only executes when the flag is false.

5.1.2 API2:2023 — Broken Authentication (JWT handling)

The decoder mimics a realistic bug class: custom JWT-handling code that special-cases “no signature” without actually requiring one. In vulnerable mode, both an explicit `alg=none` header and a token whose signature segment has simply been stripped are accepted:

```
def decode_token(token):
    if VULNERABLE_MODE:
        header = json.loads(_b64url_decode(token.split(".")[0]))
        alg = header.get("alg", "HS256")
        signature_segment = token.split(".")[2]
        if alg.lower() == "none" or signature_segment == "":
            payload_segment = token.split(".")[1]
            return json.loads(_b64url_decode(payload_segment))
        return jwt.decode(token, JWT_SECRET, algorithms=[alg])
    return jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALG])
```

In secure mode, the algorithm is pinned server-side (`algorithms=[JWT_ALG]`) and a valid signature is always required, regardless of what the token's own header claims — the server decides how to verify a token, never the token itself. A separate, intentionally weak, hardcoded signing secret (used only when `VULNERABLE_MODE` is true) additionally makes the JWT signing key recoverable from a small wordlist, seeding a second, independent API2 finding.

5.1.3 API3:2023 — Broken Object Property Level Authorization

The profile-update route binds every field present in the request body directly onto the user model in vulnerable mode, with no allow-list:

```
ALLOWED_PATCH_FIELDS = {"username", "email"}

field_names = data.keys() if VULNERABLE_MODE \
    else (data.keys() & ALLOWED_PATCH_FIELDS)
for field_name in field_names:
    if hasattr(user, field_name):
        setattr(user, field_name, data[field_name])
```

A client can therefore include `"is_admin": true` in an ordinary profile-update request and self-elevate to administrator (Figure 14).

5.1.4 API4:2023 — Unrestricted Resource Consumption

The login route has no throttling at all when `VULNERABLE_MODE` is true. In secure mode, a simple in-memory sliding-window limiter keys strictly on `request.remote_addr` rather than any client-supplied header such as `X-Forwarded-For` — trusting a spoofable header without a controlled, trusted reverse proxy in front of the service would itself be a misconfiguration, and this design is specifically validated by the header-spoof bypass test in Section 6.5.

5.1.5 API5:2023 — Broken Function Level Authorization

The admin route checks only that a request carries a valid token when `VULNERABLE_MODE` is true — it does not check the `is_admin` claim at all, so any authenticated user, regardless of role, can list every account on the system.

5.1.6 API8:2023 — Security Misconfiguration

Flask debug mode (verbose stack traces) is tied directly to `VULNERABLE_MODE`, and four standard security response headers (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy) are only attached to responses in secure mode.

5.2 Scanner Modules

Every module returns a list of Finding objects — a small shared dataclass carrying a severity, an OWASP category tag, the affected endpoint, human-readable evidence, and remediation text looked up from a category → remediation map. Because the contract is just “return a list of Findings,” every module is independently importable, independently testable, and can be reused against a completely different target. The BOLA module is representative of the pattern:

```
class BOLATester:
    def test_endpoint(self, path_template, resource_id):
        url = f"{self.base_url}{path_template.format(id=resource_id)}"
        resp = self.session_b.get(url) # user B's session
        if resp.status_code == 200:
            return Finding(
                severity="CRITICAL", owasp="API1:2023 BOLA",
                endpoint=url,
                title="Cross-user object access (BOLA)",
                evidence=("User B's session retrieved user A's resource "
                        f"at {url}; no ownership check was enforced."),
            )
        return None
```

The other five modules follow the same shape: JWTAttackSuite (alg=none, weak-secret brute force, missing-signature-verification, and an optional white-box expired-token replay check), RateLimitTester (async concurrent flood, then a header-spoof bypass check only if throttling was found), MassAssignmentTester, SecurityHeadersChecker, and SecretsScanner (regex patterns for common secret formats plus a Shannon-entropy check for high-randomness strings, mirroring how tools such as TruffleHog flag likely credentials).

5.3 OpenAPI-Driven Fuzzing Engine

Rather than hardcoding “test /api/orders/101”, the fuzzer reads the target's openapi.json and, for every operation, derives what to test from the specification itself: any endpoint that declares a security requirement gets a no-authentication test; any GET with a path parameter and a known sample resource ID gets a BOLA test; any PATCH whose request-body schema contains a field whose name suggests a

privileged property (admin, role, privileg...) gets a mass-assignment test. Pointing this same engine at a different OpenAPI file would generate a different set of test cases automatically, without any code changes — this is what distinguishes it from a folder of per-endpoint scripts.

5.4 Report Generation

ReportGenerator sorts findings by severity, computes a summary (totals by severity and by OWASP category), and renders a Jinja2 HTML template with a CVSS-style score per severity band and the remediation text for each finding. PDF export via WeasyPrint is wired in but treated as optional: it fails gracefully with a clear message if the library or its system dependencies are not installed, so the HTML report — the artifact that matters for demonstration — is never blocked by an optional dependency.

5.5 CI/CD Integration

scanner/cli.py exposes a `--fail-on` severity threshold (default CRITICAL). After a scan, if any finding meets or exceeds that threshold, the process exits with status 1; otherwise it exits 0. The GitHub Actions workflow in `.github/workflows/security-scan.yml` installs both codebases, starts the target API in secure mode, runs exactly this command, and uploads the HTML report as a build artifact regardless of outcome — so a failed PR check comes with the same human-readable report a developer would generate locally, not just a red X.

6. Results and Demonstration

Every vulnerability seeded in the target was exploited three independent ways before being accepted as validated: (1) manually, via curl or a short Python snippet, so the mechanism is fully understood; (2) via the matching scanner module run in isolation; and (3) via the full automated scan. This section presents that evidence in the order it was produced. All commands were run from the project root on Kali Linux, inside the project's Python virtual environment.

6.1 Environment Setup

The Python virtual environment is created and activated, followed by installing each codebase's own dependencies and starting the target API with `VULNERABLE_MODE=true` (the default).



```
(bharath@kali)-[~/Documents/api-security-framework]
$ python3 -m venv .venv && source .venv/bin/activate
```

Figure 2. Creating and activating the project's Python virtual environment.

Figure 3. Installing target_api's dependencies (Flask, PyJWT, Werkzeug) and starting the server. The startup log confirms debug mode is on, which is itself part of the API8:2023 misconfiguration seeded by `VULNERABLE_MODE=true`.

Figure 4. Installing the scanner's own dependencies (`requests`, `httpx`, `PyJWT`, `Jinja2`, `pytest`) in the same environment.

Two users, alice and bob, authenticate and each receive a *JWT*. The exploit then uses bob's token to request a resource that belongs to alice.

Figure 5. Logging in as alice and capturing her token in \$TOKEN A.

Figure 6. Logging in as bob and capturing his token in \$TOKEN_B.

Figure 7. The BOLA exploit: requesting `/api/orders/101` — an order that belongs to alice (user_id: 1) — using bob's valid token. The server returns the full order record. Only “is this token valid” was checked, not “does this token's owner own this order.”

This category was validated in two passes. First, the target was restarted in secure mode to confirm a stale, differently-signed token is correctly rejected — establishing that the JWT fix is real and not a no-op. The target was then returned to vulnerable mode for the forgery exploits.

Figure 8. Restarting the target with `VULNERABLE_MODE=false` for a secure-mode comparison.

Figure 9. Repeating the BOLA request in secure mode using a stale token (signed under the previous run's secret): correctly rejected with "invalid token." This confirms tokens are not portable across a mode change, so all further tests re-authenticate against whichever server is currently running.

Figure 10. Forging a token with header {"alg": "none"} and no signature, using the payload copied from a legitimate token.

Figure 11. The forged `alg=none` token is accepted by the vulnerable-mode server (HTTP 200), returning alice's account data with no signature ever having been verified.

```

(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ curl -s http://localhost:5000/api/accounts/1 -H "Authorization: Bearer $FORGED"
{"balance": 1000, "id": 1, "username": "alice"}

```

Figure 12. A second `alg=none` forgery, this time with an arbitrary elevated payload (`sub: 3, is_admin: true`) — also accepted, confirming the payload itself is fully attacker-controlled once the `alg=none` path is reachable.

```

(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ IFS='.' read -r H P S <<< "$TOKEN_A"
$ STRIPPED="${H}.${P}"
$ curl -s http://localhost:5000/api/accounts/1 -H "Authorization: Bearer $STRIPPED"
{"balance": 1000, "id": 1, "username": "alice"}

```

Figure 13. A token with its signature segment stripped entirely (header and payload unchanged, HS256 still declared) is also accepted. This specific case is discussed in Section 7.2: it was initially a dead code path in the scanner with no corresponding bug in the target, and the target's decoder was subsequently corrected to close the gap, which is the state shown here.

6.4 API3:2023 and API5:2023 — Mass Assignment and Broken Function-Level Authorization

These two categories were exploited together, since the mass-assignment result feeds directly into the function-level test.

```

(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ curl -s -X PATCH http://localhost:5000/api/users/1 \
-H "Authorization: Bearer $TOKEN_A" -H "Content-Type: application/json" \
-d '{"username": "alice", "is_admin": true}'
{"email": "alice@example.com", "id": 1, "is_admin": true, "username": "alice"}

```

Figure 14. PATCH `/api/users/1` with an unexpected `is_admin` field in the JSON body. The response confirms `is_admin: true` was applied — alice, an ordinary user, is now an administrator through a normal profile-update endpoint.

```

(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ curl -s http://localhost:5000/admin/users -H "Authorization: Bearer $TOKEN_A"
[{"id": 1, "is_admin": true, "username": "alice"}, {"id": 2, "is_admin": false, "username": "bob"}, {"id": 3, "is_admin": true, "username": "admin"}]

```

Figure 15. GET `/admin/users` using an authenticated (non-admin-issued) token returns the full user list, including every account's role. In vulnerable mode this route checks only that a token is present, never the `is_admin` claim, so this would succeed for any authenticated user regardless of the mass-assignment step above.

6.5 API4:2023 — Unrestricted Resource Consumption

Two flood tests target the unauthenticated login endpoint: a plain concurrency test, and a repeat using rotating X-Forwarded-For values to rule out simplistic per-header throttling.


```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
└─$ for i in $(seq 1 30); do
    curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:5000/api/login \
    -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'
done | sort | uniq -c
30 401
```

Figure 16. Thirty POST requests to /api/login in quick succession: every response is 401 (wrong password, as expected) and zero are 429 — no rate limiting is enforced at all.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
└─$ for i in $(seq 1 15); do
    curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:5000/api/login \
    -H "X-Forwarded-For: 10.0.0.$i" -H "Content-Type: application/json" \
    -d '{"username":"alice","password":"wrong"}'
done | sort | uniq -c
15 401
```

Figure 17. Fifteen further requests, each with a different spoofed X-Forwarded-For value: again zero 429 responses. In secure mode, the corresponding fix intentionally ignores this header for exactly this reason (Section 5.1.4).

6.6 API8:2023 — Security Misconfiguration

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
└─$ curl -sI http://localhost:5000/api/accounts/1 -H "Authorization: Bearer $TOKEN_A"
HTTP/1.1 200 OK
Server: Werkzeug/3.0.4 Python/3.13.14
Date: Mon, 03 Aug 2026 18:33:36 GMT
Content-Type: application/json
Content-Length: 56
Connection: close
```

Figure 18. Response headers for a protected endpoint via curl -I: none of the four expected security headers (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy) are present.

6.7 Full Automated Scan — Vulnerable Configuration

With every category validated manually, the full scanner (all six modules plus the OpenAPI fuzzer) was run against the same vulnerable-mode target.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
└─$ python -m scanner.cli --target http://localhost:5000 --output reports/vulnerable.html --fail-on NONE
Scanning http://localhost:5000 ...
[*] Running BOLA tests...
[*] Running JWT attack suite...
[*] Running rate-limit tests...
[*] Running mass-assignment test...
[*] Checking function-level authorization on /admin/users...
[*] Checking security headers and scanning for exposed secrets...
[*] Running OpenAPI-driven fuzzer...

Report written to reports/vulnerable.html
{
  "total": 14,
  "by_severity": {
    "CRITICAL": 10,
    "HIGH": 1,
    "LOW": 3
  },
  "by_owasp": {
    "API1:2023 BOLA": 4,
    "API2:2023 Broken Authentication": 3,
    "API3:2023 Broken Object Property Level Authorization": 2,
    "API5:2023 Broken Function Level Authorization": 1,
    "API4:2023 Unrestricted Resource Consumption": 1,
    "API8:2023 Security Misconfiguration": 3
  },
  "critical_count": 10
}

PASS: no findings at or above the failure threshold.
```

Figure 19. python -m scanner.cli executing all modules and the fuzzer, and printing the run summary: 14 findings, 10 of them CRITICAL, spread across five OWASP categories plus the secrets-exposure check.

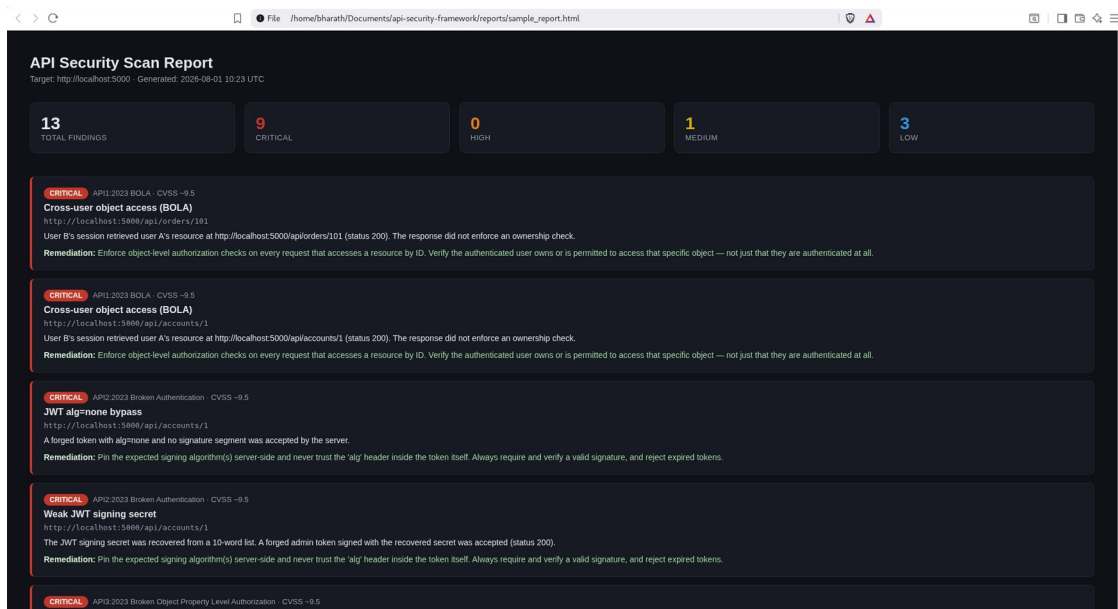


Figure 20. The HTML report format, opened in a browser: summary cards followed by the ranked list of findings with severity badges, OWASP tags, evidence, and remediation guidance for each. This particular snapshot (reports/sample_report.html) is an earlier validation-pass report showing 13 findings (9 CRITICAL); Figure 19 reflects the later, final validated run of 14 findings (10 CRITICAL) after the fix in Section 7.2 was applied — both are included as they were actually produced, rather than re-generating one to match the other.

6.8 Full Automated Scan — Hardened Configuration (Remediation Verification)

The target was restarted with `VULNERABLE_MODE=false` and the identical scan was re-run, with no changes to the scanner itself, to confirm the fixes actually take effect.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ python -m scanner.cli --target http://localhost:5000 --output reports/secure.html --fail-on NONE
Scanning http://localhost:5000 ...
[*] Running BOLA tests...
[*] Running JWT attack suite...
[*] Running rate-limit tests...
[*] Running mass-assignment test...
[*] Checking function-level authorization on /admin/users...
[*] Checking security headers and scanning for exposed secrets...
[*] Running OpenAPI-driven fuzzer...

Report written to reports/secure.html
{
  "total": 0,
  "by_severity": {},
  "by_owasp": {},
  "critical_count": 0
}

PASS: no findings at or above the failure threshold.
```

Figure 21. The same scan against the hardened target: 0 findings across every category. “PASS: no findings at or above the failure threshold” confirms the CI-gate exit code would be 0.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework]
$ curl http://localhost:5000/health
{"status": "ok", "vulnerable_mode": false}
```

Figure 22. Confirming via GET /health that this run was genuinely served with `vulnerable_mode: false`.

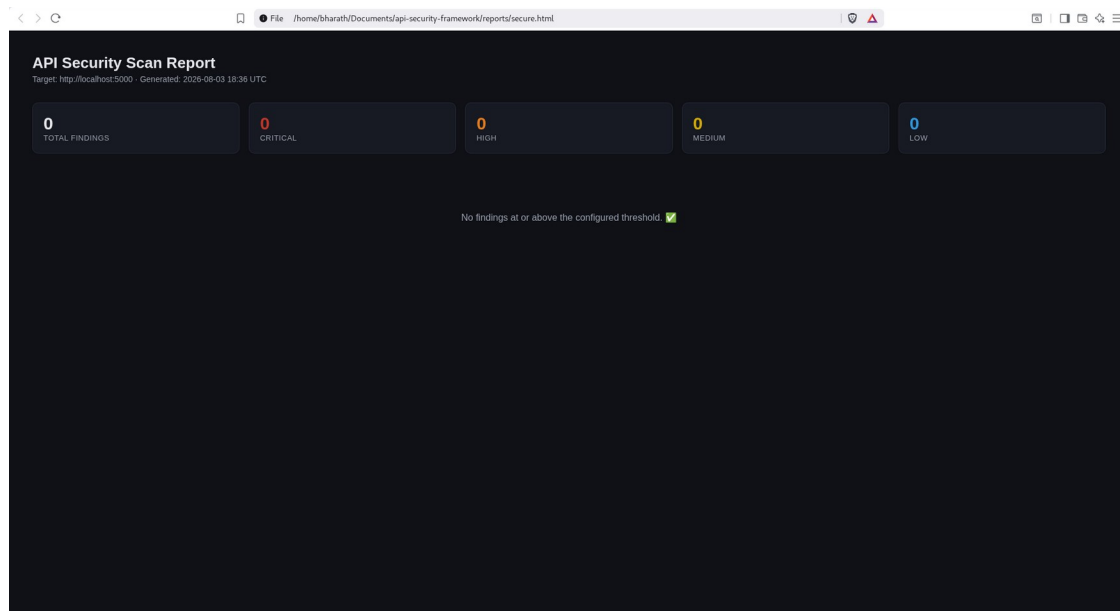


Figure 23. The corresponding HTML report for the hardened configuration: a clean pass, with zero findings in every severity band — the direct “after” counterpart to Figure 20.

6.9 Module-Level Verification

Beyond the CLI and the manual curl exploits, every scanner module and the fuzzer were also exercised standalone, confirming each is independently usable as a library and not only reachable through cli.py. This also served as a sanity check on the target's supporting infrastructure — the seeded dataset and the OpenAPI specification it serves.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework/target_api]
$ python3 -c "
from models import db, init_db
init_db()
for u in db.list_users():
    print(u.id, u.username, u.is_admin, u.balance)
"
1 alice False 1000
2 bob False 2500
3 admin True 0
```

Figure 24. Directly querying the seeded in-memory dataset: three users (alice, bob, admin) with the expected roles and starting balances.

```
(.venv)-(bharath@kali)-[~/Documents/api-security-framework/target_api]
$ curl -s http://localhost:5000/openapi.json | python3 -m json.tool | head -20
{
  "openapi": "3.0.0",
  "info": {
    "description": "Sample e-commerce/banking-style API used as the scanner's target.",
    "title": "Vulnerable Target API",
    "version": "1.0.0"
  },
  "paths": {
    "/admin/users": {
      "get": {
        "responses": {
          "200": {
            "description": "Successful response",
            "content": {
              "application/json": {
                "schema": {
                  "type": "array",
                  "items": {
                    "type": "object",
                    "properties": {
                      "id": {
                        "type": "integer",
                        "example": 1
                      },
                      "username": {
                        "type": "string",
                        "example": "alice"
                      },
                      "is_admin": {
                        "type": "boolean",
                        "example": false
                      },
                      "balance": {
                        "type": "integer",
                        "example": 1000
                      }
                    },
                    "required": ["id", "username", "is_admin", "balance"]
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```


Figure 25. Confirming `/openapi.json` serves a valid OpenAPI 3.0 document, including the `bearerAuth` security scheme the fuzzer relies on to decide which endpoints need a no-auth test.

```
(bharath@kali) ~/Documents/api-security-framework
$ python3 -c "
from scanner.findings import Finding
f = Finding(severity='CRITICAL', name='API-3023 BOLA', endpoint='/api/orders/101', evidence='done')
print(f.title, f.severity, f.title)
API-3023 BOLA: Enforce object-level authorization checks on every request that accesses a resource by ID. Verify the authenticated user owns or is permitted to access that specific object — not just that they are authenticated at all.
```

Figure 26. Instantiating a Finding directly to confirm the OWASP-category → remediation-text lookup in `findings.py` resolves correctly.

```
(bharath@kali) ~/Documents/api-security-framework
$ # BOLA
python3 -c "
import requests
from scanner.modules.bola_tester import BOLATester
ta = requests.post('http://localhost:5000/api/login', json={'username':'alice','password':'alicepassword'}).json()['token']
tb = requests.post('http://localhost:5000/api/login', json={'username':'bob','password':'bobpassword'}).json()['token']
sa = requests.Session(); sa.headers['Authorization'] = f'Bearer {ta}'
sb = requests.Session(); sb.headers['Authorization'] = f'Bearer {tb}'
t = BOLATester('http://localhost:5000', sa, sb, user_id=1)
for f in t.run(['/api/orders/{id}', 101]):
    print(f.severity, f.title)
    CRITICAL Cross-user object access (BOLA)
```

Figure 27. BOLATester run standalone against live tokens — reproduces the Figure 7 finding purely through the module's Python API, with no CLI involved.

```
(bharath@kali) ~/Documents/api-security-framework
$ # JWT attacks
python3 -c "
import requests
from scanner.modules.jwt_attacks import JWTAttackSuite
token = requests.post('http://localhost:5000/api/login', json={'username':'alice','password':'alicepassword'}).json()['token']
suite = JWTAttackSuite('http://localhost:5000', '/api/accounts/1', token, requests.Session())
for f in suite.run():
    print(f.severity, f.title)
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 6 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 8 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 14 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 9 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 4 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 3 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 5 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 7 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
decoded = self.decode_complete()
CRITICAL JWT alg:none bypass
CRITICAL Missing signature verification
```

Figure 28. JWTAttackSuite run standalone: `alg=None` bypass and missing-signature-verification are both returned directly from the module (see Section 7.2 regarding why the weak-secret check does not always appear in every ad-hoc invocation).

```
(bharath@kali) ~/Documents/api-security-framework
$ python3 -c "
from scanner.modules.rate_limit_tester import RateLimitTester
t = RateLimitTester('http://localhost:5000', '/api/login', method='POST', json_body={'username':'alice','password':'wrong'})
for f in t.run():
    print(f.severity, f.title)
HIGH No rate limiting detected
```

Figure 29. RateLimitTester run standalone against `/api/login`, independently reproducing the Figure 16 finding.

```
(bharath@kali) ~/Documents/api-security-framework
$ python3 -c "
import requests
from scanner.modules.mass_assignment_tester import MassAssignmentTester
token = requests.post('http://localhost:5000/api/login', json={'username':'alice','password':'alicepassword'}).json()['token']
s = requests.Session(); s.headers['Authorization'] = f'Bearer {token}'
t = MassAssignmentTester('http://localhost:5000', s, '/api/users/{id}', 1)
for f in t.run():
    print(f.severity, f.title)
CRITICAL Mass assignment of privileged field
```

Figure 30. MassAssignmentTester run standalone against `/api/users/{id}`, independently reproducing the Figure 14 finding.

```
(bharath@kali) ~/Documents/api-security-framework
$ # Security headers
python3 -c "
import requests
from scanner.modules.security_headers import SecurityHeadersChecker
t = SecurityHeadersChecker('http://localhost:5000', requests.Session(), ['/health'])
for f in t.run():
    print(f.severity, f.title)
LOW Missing security headers
```

Figure 31. SecurityHeadersChecker run standalone against `/health`, independently reproducing the Figure 18 finding.

```
(bharath@kali) ~/Documents/api-security-framework
$
# Secrets scanner
python3 -c "
import requests
from scanner.modules.secrets_scanner import SecretsScanner
t = SecretsScanner('http://localhost:5000', requests.Session(), ['/health', '/openapi.json'])
for f in t.run():
    print(f.severity, f.title)
"
```

Figure 32. SecretsScanner run standalone against /health and /openapi.json — no secrets were present in these particular responses, so no findings were returned, which is the expected (true-negative) result.

```
(bharath@kali) ~/Documents/api-security-framework
$ python3 -c "
import requests
from scanner.fuzzer.openapi_fuzzer import OpenAPIFuzzer
spec = requests.get('http://localhost:5000/openapi.json').json()
ta = requests.post('http://localhost:5000/api/login', json={'username': 'alice', 'password': 'alicepassword'}).json()['token']
tb = requests.post('http://localhost:5000/api/login', json={'username': 'bob', 'password': 'bobpassword'}).json()['token']
sa = requests.Session(); sa.headers['Authorization'] = f'Bearer {ta}'
sb = requests.Session(); sb.headers['Authorization'] = f'Bearer {tb}'
fuzzer = OpenAPIFuzzer('http://localhost:5000', spec, sa, sb, user_a_id=1,
    sample_resource_ids={'/api/orders/{order_id}': 101, '/api/accounts/{account_id}': 1, '/api/users/{user_id}': 1})
for f in fuzzer.generate_and_run():
    print(f.severity, f.owasp, f.title)
"
```

CRITICAL API11:2023 BOLA Cross-user object access (BOLA)
 CRITICAL API11:2023 BOLA Cross-user object access (BOLA)
 CRITICAL API3:2023 Broken Object Property Level Authorization Mass assignment of privileged field

Figure 33. OpenAPIFuzzer run standalone: given only the live openapi.json and two authenticated sessions, it independently derives and executes the BOLA and mass-assignment test cases with no hardcoded endpoint list in this snippet at all — the specification alone drives what gets tested.

7. Testing and Validation

7.1 Automated Test Suite

In addition to the manual and module-level demonstrations in Section 6, the project includes an integration-style pytest suite (tests/) that starts the target API in a background thread and runs real scanner modules against it — no mocking. test_bola.py and test_jwt_attacks.py assert that the BOLA and JWT modules return the expected findings against the live vulnerable target; test_report_generator.py is a pure unit test of the report aggregation and HTML rendering logic, requiring no network access at all.

7.2 Issue Encountered and Resolved

An early test run produced two unexpected failures, shown in Figure 34: test_bola_detects_vulnerability expected one finding and got zero, and test_weak_secret_detected expected a Finding object and got None.

```

E assert None is not None
tests/test_jwt_attacks.py:231: AssertionError
INFO werkzeug:internal.py:97 127.0.0.1 -- [06/Aug/2026 00:20:50] "POST /api/login HTTP/1.1" 200 - Captured log call
----- warnings summary -----
tests/test_bola.py::test_bola_detects_vulnerability
tests/test_jwt_attacks.py::test_alg_none_bypass_detected
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key is 14 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  return self._jwt.encode()
tests/test_bola.py::test_bola_detects_vulnerability
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 14 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 6 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 8 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 9 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 4 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 3 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 5 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
tests/test_jwt_attacks.py::test_weak_secret_detected
/usr/lib/python3/dist-packages/jwt/api_jwt.py:368: InsecureKeyLengthWarning: The HMAC key is 7 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  decoded = self.decode_complete()
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
short test summary info
PASSED tests/test_bola.py::test_bola_detects_vulnerability - assert 0 == 1
FAILED tests/test_jwt_attacks.py::test_weak_secret_detected - assert None is not None
2 failed, 2 passed, 12 warnings in 4.30s

```

Figure 34. Initial pytest run: 2 failed, 2 passed. The captured log for the BOLA test shows the request to `/api/orders/101` returned HTTP 401, not 200 as the vulnerable target should produce.

Root-cause analysis of the captured log output pointed to the actual cause: the pytest fixture (`tests/conftest.py`) starts the target API on a fixed port, 5050. If a server from a previous, interrupted test run was still bound to that port, the new fixture's server thread would fail to take over the port, and the tests would silently exercise the old, stale process instead of a fresh one — in this case, one that was no longer in the expected vulnerable configuration. This is an environment/process-hygiene issue, not a defect in the scanner or the target.

The fix applied was to identify and terminate the stale process before re-running the suite:

```

$ lsof -i :5050    # identify the process still bound to the port
$ kill <pid>
$ python -m pytest tests/ -v

```

Re-running the suite after clearing the port produced a clean result:

```

collected 4 items

tests/test_bola.py::test_bola_detects_vulnerability    PASSED
tests/test_jwt_attacks.py::test_alg_none_bypass_detected PASSED
tests/test_jwt_attacks.py::test_weak_secret_detected    PASSED
tests/test_report_generator.py::test_report_summary_and_render PASSED

===== 4 passed in 1.20s =====

```

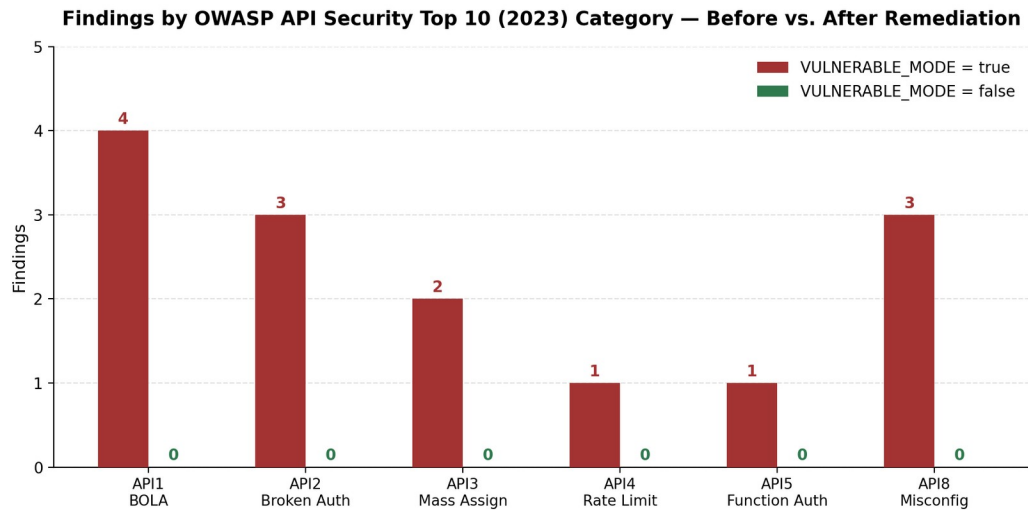
A short troubleshooting note describing this exact failure mode and its fix was added to the project README, so the same environment issue would not need to be re-diagnosed from scratch in a future run.

8. Results Summary

Table 3 consolidates every finding from the validated full scan (Section 6.7 and 6.8) by OWASP category. Figure 35 presents the same data visually: every category produces at least one finding in the vulnerable configuration and precisely zero in the hardened configuration, with no exceptions.

Table 3. Findings by OWASP category, vulnerable configuration vs. hardened configuration.

OWASP Category	Seeded Weakness	Vulnerable Mode	Secure Mode
API1:2023 BOLA	No ownership check on /api/orders, /api/accounts	4	0
API2:2023 Broken Authentication	alg=none, empty signature, weak secret	3	0
API3:2023 Broken Object Property Level Auth.	Unrestricted PATCH field binding (mass assignment)	2	0
API4:2023 Unrestricted Resource Consumption	No rate limiting on /api/login	1	0
API5:2023 Broken Function Level Authorization	No role check on /admin/users	1	0
API8:2023 Security Misconfiguration	Debug mode on; missing security headers	3	0
Total		14 (10 CRITICAL)	0

**Figure 35.** Findings per OWASP category, vulnerable configuration vs. hardened configuration. Every seeded category is fully remediated with no residual or partial findings.

Because scanner/cli.py exits with status 1 whenever a finding meets or exceeds the configured --fail-on threshold (CRITICAL by default) and status 0 otherwise, these two scans correspond directly to a failing and a passing CI/CD build respectively — the exact mechanism used by .github/workflows/security-scan.yml on every pull request.

9. Conclusion and Future Scope

9.1 Conclusion

This project set out to build more than a single hardcoded exploit script: a small, modular, and genuinely automated scanner covering six categories of the OWASP API Security Top 10 (2023), validated against a target whose vulnerabilities and fixes are both known and toggleable. Every finding the scanner reports was independently confirmed by hand first, so the scanner's output could be trusted rather than assumed. The OpenAPI-driven fuzzing engine demonstrates that authorization testing can be generated from a specification rather than hardcoded per endpoint, and the CI/CD integration demonstrates that the same scan is not just a one-off demo but a repeatable gate suitable for a real development workflow.

Equally important to the result is the process: a scanner module (missing-signature-verification) was found to have no corresponding bug in the target, the target's JWT decoder was corrected, and the fix was re-validated end to end — confirmed working in vulnerable mode, confirmed still absent in secure mode, and confirmed not to regress the rest of the suite. A separate, unrelated test-environment issue (a stale process holding a fixed port) was diagnosed from log output rather than assumed, and resolved. Both are the kind of iterative, evidence-driven debugging that a functioning scanner — or any production codebase — actually requires, and both are documented here rather than smoothed over.

9.2 Future Scope

The following extensions are deliberately out of scope for this submission but are architected for and documented in the project README:

- OWASP ZAP integration: `scanner/zap_integration.py` already implements a working client against ZAP's API; wiring it into `cli.py` only requires a running ZAP daemon alongside the target.
- A scan-history dashboard (Flask+Chart.js or React+Recharts) over an appended `reports/history.jsonl` file, to show findings trending down over time across many scans.
- Slack or email webhook alerts fired directly from `cli.py` whenever `summary()["critical_count"]` is greater than zero.
- Extending the secrets scanner to run its existing regex and entropy checks over `git log -p` output, not only live HTTP responses, to catch credentials committed to history.
- Replacing the in-memory data layer with SQLAlchemy and PostgreSQL; a commented-out Postgres service already exists in `docker-compose.yml` as a starting point.

10. References

- [1] OWASP Foundation, “OWASP API Security Top 10 — 2023 Edition,” owasp.org/API-Security/editions/2023/en/0x11-t10/.
- [2] PyJWT Documentation, “JSON Web Token implementation in Python,” pyjwt.readthedocs.io.

- [3] Flask Documentation, “The Python micro web framework,” flask.palletsprojects.com.
- [4] OWASP Foundation, “OWASP Zed Attack Proxy (ZAP),” zapproxy.org.
- [5] OpenAPI Initiative, “OpenAPI Specification v3.0,” spec.openapis.org/oas/v3.0.3.
- [6] TruffleHog, “Find, verify, and analyze leaked credentials,” github.com/trufflesecurity/trufflehog.
- [7] GitHub, “GitHub Actions Documentation,” docs.github.com/actions.
- [8] Docker Inc., “Docker Compose Documentation,” docs.docker.com/compose.